RippleLab Day 09 Progress Report

Day: 09 of 30

Date: 2026-08-15

Status: Complete

Branch: `codex/day-09-inflation`

Implementation commit: `95bc599`

Deployed commit: `95bc599`

Production version: 5

Quality gate: Full local suite passed

1. Objective

Build RippleLab's first personal inflation engine. Day 9 must compare a current and target inflation path, translate the headline rate through an editable household expense basket, show which categories drive the result, calculate salary and portfolio purchasing-power diagnostics, and return an inspectable causal graph without delegating money calculations to an LLM.

2. Acceptance Criteria

- [x] Add a canonical `inflation_change` request example.
- [x] Start from the signed-in user's saved income, expense and asset profile.
- [x] Create an editable five-category monthly expense basket.
- [x] Keep category pass-through values explicit and editable.
- [x] Compare current and target inflation paths deterministically.
- [x] Calculate annualized category and total expense impact.
- [x] Calculate personal weighted inflation.
- [x] Calculate real salary growth and real portfolio return.
- [x] Keep diagnostics separate from the headline expense total.
- [x] Return assumptions, evidence, confidence, warnings and causal graph data.
- [x] Reuse the Day 8 graph inspector for the inflation topology.
- [x] Cover API, high/low inflation, salary growth, desktop, mobile and accessibility cases.
- [x] Update GitHub, the PDF progress record and the owner-only deployment.

3. Delivered User Experience

A signed-in user opens the Inflation simulator from the dashboard or navigation. RippleLab creates a transparent starting basket from the saved profile and allows the user to edit every monthly amount and category transmission value before calculation.

```
Current headline CPI + selected change
  -> five category inflation rates
  -> personal expenditure-weighted inflation
```

```
-> current-versus-target annual expense impact
-> real salary and portfolio diagnostics
-> inspectable causal graph
```

The result shows the target headline rate, the household's weighted rate, projected monthly expenses, annual scenario impact, real salary growth, real portfolio return and a category-by-category driver table. A deterministic sensitivity range shows how the expense result changes when all category transmission assumptions move together.

4. Personal Basket Design

The v1 profile stores total essential expenses, discretionary expenses and rent rather than a detailed consumption ledger. The frontend therefore creates an editable starting allocation:

Category	Starting allocation	Default pass-through
Food and groceries	40% of essential expenses	100%
Transport	20% of essential expenses	120%
Utilities	20% of essential expenses	80%
Housing	Saved monthly rent	100%
Other spending	Remaining essential plus discretionary expenses	70%

These defaults are model choices, not observed household facts. Every value is visible and editable. EMI is excluded because it is a financial payment already handled by the repo-rate model rather than consumption spending.

5. Deterministic Model

Category projection

```
category rate = headline inflation x category pass-through
projected spend = monthly spend x (1 + category rate x months / 12)
annual impact = -(target projected spend - baseline projected spend) x 12
```

Formula version: `inflation.category_projection.v1`

Personal inflation

```
personal rate = sum(category spend x category rate) / total spend
```

Formula version: `inflation.personal_basket.v1`

Real purchasing-power growth

```
real growth = (1 + nominal growth) / (1 + personal inflation) - 1
```

Formula version: `inflation.real_growth.v1`

The engine uses Decimal arithmetic and rounds money to integer paise with `ROUND_HALF_UP`. Salary growth and nominal portfolio return are explicit assumptions. Their diagnostics are excluded from `annualNetImpactPaise` so the product does not combine income purchasing power, asset-value change and expense pressure into a misleading total.

6. Known-value Result

The seeded salaried Bengaluru household has an INR 80,000 monthly basket. The current headline rate is 4%, the selected increase is 2 percentage points and the target is therefore 6%.

Output	Verified value
Personal weighted inflation	5.19%
Target projected monthly basket	INR 84,152.00
Annual impact versus current path	-INR 16,608.00
Food annual effect	-INR 5,280.00
Transport annual effect	-INR 3,168.00
Utilities annual effect	-INR 2,112.00
Other-spending annual effect	-INR 6,048.00
Real salary growth at 5% nominal growth	approximately -0.18%
Real portfolio return at 7% nominal return	approximately 1.72%

Changing food pass-through from 100% to 50% changes the annual expense impact to -INR 13,968.00 without changing the saved profile.

7. Sensitivity and Confidence

The uncertainty range is a deterministic sensitivity check. The engine reduces every category pass-through by 20 percentage points, recalculates the result, then increases every pass-through by 20 percentage points and recalculates again. Each value is clamped between 0% and 200%.

The shared result contract retains `p10`, `p50` and `p90` field names, but Day 9 explicitly labels the method `deterministic_bounds`. The range is not a probability distribution and is not a forecast of future inflation.

Expense confidence is medium. Monthly category amounts are personalized and the arithmetic is deterministic, while the future headline path and category transmission remain selected assumptions. Salary and portfolio confidence is also medium because future nominal growth rates are not observed outcomes.

8. Evidence

Day 9 uses primary official sources:

- Ministry of Statistics and Programme Implementation, National Metadata Structure for Consumer Price Index - expenditure weighting and CPI methodology.
- Reserve Bank of India, Monetary Policy Framework - headline CPI inflation target context.

The sources support the economic context and weighting approach. They do not validate the user's category pass-through choices. No current inflation observation is hard-coded; the current and target rates remain explicit scenario inputs.

9. Causal Graph

The engine returns ten nodes and thirteen edges:

```
Target headline CPI
  -> Food prices
  -> Housing prices
```

```
-> Transport prices
-> Utilities prices
-> Other prices
-> Personal basket inflation
-> Projected monthly expenses
-> Real salary growth
-> Real portfolio return
```

Each headline-to-category edge references its pass-through assumption. Each category-to-basket edge references the entered monthly spend. The final outcome links reference the appropriate salary or portfolio assumption. Mechanisms, evidence IDs, lag ranges and confidence arrive in the FastAPI result.

The existing React Flow inspector now supports market and outcome node categories, an inflation-specific layout and formula references while preserving the repo-rate graph. Nodes and edges remain read-only, pointer- and keyboard-selectable, responsive and source-backed.

10. API and Contract

`POST /v1/simulations/inflation` accepts the canonical `SimulationRequest` with `scenario.type = inflation_change` and returns the shared `SimulationResult` model version `1.0.0`.

The protected Next.js route requires a signed-in session and forwards the request to the server-only economic engine address. FastAPI/Pydantic rejects missing, incorrectly typed or out-of-range assumptions with a human-readable `INVALID_INFLATION_ASSUMPTION` response. The browser never receives the engine address.

The canonical `inflation-request.v1.json` fixture is validated against the same JSON Schema that generates TypeScript types and defines the Python mirror.

11. Test and Verification Evidence

Check	Result
Economic-engine API and calculation suite	45 passed
Day 9 inflation engine tests	6 passed
Python lint and formatting	Passed
Canonical contracts and inflation request fixture	Passed
Profile ownership, RLS, money and consent assertions	Passed
Repository secret scan	Passed
ESLint, generated routes and TypeScript	Passed
Next.js production build	Passed; 19 routes generated
Complete desktop and mobile browser suite	27 passed; 7 intentional device skips
Inflation result-state accessibility	No serious or critical violations
Mobile graph and horizontal-overflow check	Passed
Sites Vinext production bundle	Passed
Sites production deployment	Version 5 succeeded

The Python suite reports one existing dependency deprecation warning from FastAPI's Starlette test client regarding `httpx`. It does not affect the 45 passing tests.

Secret scanning now skips generated build, coverage, browser-result and Wrangler folders, avoiding false positives from minified React Flow CSS while continuing to scan source files.

12. Production Deployment

The Day 9 progress build is live at ripplelab-progress.nayaksiddartha397.chatgpt.site. Sites version 5 was built, saved and deployed from exact pushed commit `95bc599`. Access remains owner-only during development.

The landing page, progress page, design system and other read-only product surfaces are operational. The authenticated inflation and repo-rate simulators are complete and verified locally, but hosted account/profile/simulation execution still requires production Supabase values and a hosted FastAPI endpoint. No placeholder credentials were committed.

13. Important Files

- `services/economic-engine/src/ripplelab_engine/inflation.py` and `test_inflation.py` - deterministic model plus known-value, sensitivity, validation and API coverage.
- `apps/web/src/lib/scenarios/inflation.ts` and `inflation-simulator.tsx` - profile-derived request, editable form, outputs and category drivers.
- `apps/web/src/app/api/simulations/inflation/route.ts` - authenticated server-to-engine boundary.
- `causal-graph-explorer.tsx` and `inflation.spec.ts` - reusable graph presentation plus desktop, mobile and accessibility checks.
- `inflation-request.v1.json` and `INFLATION_ENGINE_V1.md` - canonical fixture and model reference.
- `apps/web/public/og-day9.png` - Day 9 social-preview card.

14. Decisions and Limitations

- Compare current and target headline paths; do not attribute all target inflation to the scenario change.
- Use personal expenditure weights and editable pass-through instead of implying national or city-level precision.
- Keep salary and portfolio diagnostics outside the headline expense total.
- Use linear horizon scaling in v1 and exclude EMI from the consumption basket.
- Exclude other investments because the shared profile snapshot does not yet carry that field.
- Keep sensitivity deterministic; Monte Carlo remains a later checkpoint.
- Do not model substitution, taxes, policy reactions or behavioral spending changes.
- Continue treating results as educational simulations rather than forecasts or financial advice.

No code blocker remains for Day 9.

15. Day 10 Handoff

Day 10 should add the oil-price engine. It should separate crude-price inputs from retail fuel assumptions, calculate direct fuel effects and explicit indirect pass-through to transport, food and utilities, show why renter and commuting profiles differ, and return the same stable result and causal-graph envelope.

Sign-off: Prepared by Codex | Evidence reviewed: Yes | Ready to continue: Yes